# CO1 Assignment 1 – Worked Study Guide

Reliable Billing Program for a Rural Electricity Co-operative

**Important:** This is a worked learning guide rather than a submission-ready answer. Use the explanations, calculations, and checkpoints to write and test your own final submission.

## TASK 1 – Understand the Problem and Identify the Data

**Inputs**

• Previous meter reading – the earlier register value.

• Current meter reading – the latest register value.

• Household number – identifies the consumer.

• Category – identifies the applicable customer category, if used by the final design.

**Outputs**

• Units consumed

• Energy charge

• Fixed charge

• Total bill

**Constants and rules**

• The meter is a 16-bit unsigned register: 0 to 65,535.

• If the current reading is lower than the previous reading, the register has rolled over.

• The bill must not become negative.

• The total must equal fixed charge + energy charge.

• The tariff should be defined in one place so that a revision can be applied consistently.

**Rollover calculation:** For previous = 65,400 and current = 120, the register range is 65,536 values. Consumption = (65,536 − 65,400) + 120 = **256 units**.

**Text versus integer readings:** storing a reading such as 01530 as text needs digit conversion before arithmetic. Conversion takes time proportional to the number of digits, while an integer stores the value directly.

## TASK 2 – Choose the Data Types

| Data | Suggested type | Reason |
|---|---|---|
| Meter readings | uint16_t / unsigned short | Matches the 0–65,535 meter range. |
| Units consumed | unsigned integer type | Consumption is non-negative. |
| Money in paise | long / long long | Exact integer arithmetic avoids floating-point rounding. |
| Household number | unsigned integer type | Identifier is non-negative. |
| Category | enum or integer | Represents a small fixed set of categories. |

**16-bit rollover:** an unsigned 16-bit value wraps from 65,535 back to 0. Thus 65,400 → 120 represents 256 units.

**Why not float for money?** Binary floating-point cannot represent many decimal fractions exactly. Using paise as integers makes calculations exact; ■2.25 is 225 paise.

**4000-bill precision example:** repeated floating-point calculations can accumulate tiny representation errors. Integer paise avoids that.

Task 2 checkpoints: consumption(1200, 1530) should be 330 units; consumption(65400, 120) should be 256 units. The first print should show a 2-byte meter type and maximum reading 65535.

## TASK 3 – Define the Constants

Use the named macros supplied by the assignment so the tariff is maintained in one place.

• First slab: first 100 units.

• Second slab: units 101–200 at 225 paise per unit.

• Third slab: units above 200 at 450 paise per unit.

• Fixed charge: 5000 paise (■50).

**Slab calculation:** start with the fixed charge; add the applicable energy charge for the first slab, then the second slab, then the remaining units at the third rate.

**Expected checks:**

• 0 units → ■50.00

• 100 units → ■50.00

• 150 units → ■162.50

• 200 units → ■275.00

• 350 units → ■950.00

Changing RATE2 from 225 to 230 should consistently change every bill affected by the second slab.

**#define vs const vs enum:** #define is handled by the preprocessor; const gives a typed object; enum is useful for named integral choices. For this task, follow the instruction to use the provided macros.

## TASK 4 – Use Macros Safely

Fully parenthesize macro arguments and the complete expression so operator precedence cannot change the intended result.

An unsafe square macro can produce the wrong result when given an expression such as a + 1. The safe pattern concept is to parenthesize both the argument and the complete multiplication expression.

MAX(a,b) can evaluate an argument more than once. Therefore an argument with a side effect, such as an increment, can behave unexpectedly. A static inline function is often safer because it gives normal argument evaluation and type checking.

**Complexity:** these simple calculations are O(1) time and O(1) extra space per use. Macro expansion can increase generated code size when used many times; an inline function keeps one function definition and may be inlined by the compiler.

## FINAL VALIDATION CHECKLIST

■ Compile with GCC and warnings enabled.

■ Run every test case shown in the assignment.

■ Verify the rollover case gives 256, not a negative number.

■ Keep money calculations in paise.

■ Verify all five Task 3 outputs.

■ Change RATE2 once and check affected results.

■ Keep separate folders for each task and include source, evidence, and README as required.